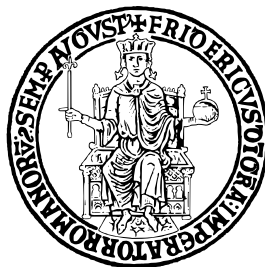


UNIVERSITÀ DEGLI STUDI DI NAPOLI FEDERICO II



SCUOLA POLITECNICA E DELLE SCIENZE DI BASE

DIPARTIMENTO DI INGEGNERIA ELETTRICA E TECNOLOGIE DELL'INFORMAZIONE

CORSO DI LAUREA TRIENNALE IN INFORMATICA

PROGETTO DI INGEGNERIA DEL SOFTWARE

DIETIESTATES25

Studenti

Luca Barrella
Fabrizio Apuzzo
Maria Della Valle

Professori

Sergio Di Martino
Luigi Libero Lucio Starace

Anno Accademico 2024–2025

Indice

0	Introduzione	5
0.1	Struttura del Documento	5
1	Analisi e Specifica dei Requisiti Software	7
1.1	Individuazione dei Target	7
1.1.1	Clienti	7
1.1.2	Agenti immobiliari	9
1.1.3	Amministratori	10
1.2	Modellazione dei casi d'uso	10
1.2.1	Descrizione dei casi d'uso	10
1.2.2	Diagramma dei casi d'uso	11
1.3	Descrizione dei requisiti non funzionali e di dominio	13
1.3.1	Requisiti non funzionali	13
1.3.2	Requisiti di dominio	14
1.4	Esempi di casi d'uso	14
1.4.1	Caso d'uso #1:	15
1.4.2	Caso d'uso #2:	16
1.4.3	Caso d'uso #3:	17
1.4.4	Caso d'uso #4:	18
1.5	Glossario	18
2	Progettazione del Sistema	21
2.1	Obiettivi di Progettazione	21
2.2	Architettura del Sistema	21
2.2.1	Front-End	22
2.2.2	Back-End	23
2.2.3	Database	25
3	Testing e Valutazione dell'Usabilità	27
3.1	Test Automatici	27
3.1.1	Metodo 1: <code>createPropertyWithImages</code>	28
3.1.2	Metodo 2: <code>getAgentOffers</code>	29
3.1.3	Metodo 3: <code>calculateBoundingBox</code>	30
3.1.4	Metodo 4: <code>getAgentVisits</code>	31
3.2	Valutazione dell'Usabilità	32
3.2.1	Expert Review (Ispezione Euristica)	32
3.2.2	Test con Utenti	33
3.3	SonarQube	35
3.4	Accessibilità	35

A Evidenza di versioning	37
--------------------------	----

Capitolo 0

Introduzione

DietiEstates25 è una piattaforma dedicata alla gestione di servizi immobiliari che consente alle agenzie di pubblicare annunci di proprietà ed agli utenti di visualizzare gli immobili disponibili ed effettuare offerte di acquisto o locazione.

Il presente documento illustra dettagliatamente il processo di sviluppo del sistema DietiEstates25 dall'analisi dei requisiti, alle scelte progettuali, per concludersi con i test di verifica all'implementazione e con la valutazione dell'usabilità della piattaforma.

0.1 Struttura del Documento

Il documento è organizzato in tre capitoli che seguono il processo di sviluppo del sistema DietiEstates25:

- Il **Capitolo 1 - Analisi e Specifica dei Requisiti Software** presenta un'analisi delle funzionalità del sistema attraverso diagrammi UML, descrizioni testuali strutturate secondo i template di A. Cockburn e prototipi delle interfacce utente. Include inoltre una definizione degli utenti target e modelli di dominio per la formalizzazione del problema.
- Il **Capitolo 2 - Progettazione del Sistema** illustra le scelte architetture e le tecnologie adottate, con l'ausilio di notazioni UML. Particolare attenzione è dedicata alla progettazione delle interfacce utente e alla motivazione delle scelte progettuali effettuate.
- Il **Capitolo 3 - Testing e Valutazione dell'Usabilità** presenta la fase di verifica del sistema attraverso test automatici. Include inoltre una valutazione dell'usabilità condotta mediante ispezioni sistematiche e test con utenti reali, completata da un'analisi dei risultati dei questionari di valutazione.

Capitolo 1

Analisi e Specifica dei Requisiti Software

Il primo passo per la creazione del sistema DietiEstates25 è l'identificazione dei requisiti con successiva analisi. Quest'ultima è strutturata partendo dalla definizione degli utenti target. Segue la modellazione dei casi d'uso con l'ausilio di Use Case Diagram, descrizioni testuali e prototipi rapidi di interfacce utente.

1.1 Individuazione dei Target

Un sistema di annunci immobiliari come DietiEstates25 può avere come target diversi gruppi di utenti. Sono state identificate però tre distinte categorie principali, ognuna con specifiche necessità e livelli di accesso alle funzionalità della piattaforma:

- Clienti
- Agenti immobiliari
- Amministratori

Segue una definizione di ciascuno di essi, accompagnata da esempi di utenti tipici di ognuna delle categorie, illustrati tramite *Personas*.

1.1.1 Clienti

Utilizzatori principali dei servizi offerti da DietiEstates25, i clienti si configurano come il gruppo più ampio degli utenti. Si tratta di soggetti interessati all'acquisto di immobili, motivati dall'opportunità di ottenere prezzi vantaggiosi tramite il sistema di offerte. Possono appartenere a qualsiasi fascia di età o classe economica e ognuno ha esigenze differenti.

Figura 1.1: Esempio di *Persona* rappresentativa di una clienteFigura 1.2: Esempio di *Persona* rappresentativa di un cliente

1.1.2 Agenti immobiliari

Un altro segmento significativo dell'utenza della piattaforma è rappresentato dagli agenti immobiliari, professionisti del settore, ingaggiati dalle agenzie che hanno scelto DietiEstates25. Dotati di una conoscenza approfondita degli immobili loro affidati, questi operatori interagiscono con i potenziali clienti attraverso il sistema di gestione delle offerte.



The infographic features a circular portrait of a woman with long dark hair wearing a blue top. To the right of the portrait, there are three sections: 'BIO', 'GOAL', and 'INTERESSI'. Below the portrait, the name 'Silvana' is written in a large, bold font, with 'Agente Immobiliare' in a smaller font underneath. The background is a light teal color with a dark teal curved shape at the bottom left.

BIO

Intraprendente new-entry con obiettivi chiari. Fortemente determinata a sfoggiare il suo talento nel mondo delle vendite.

GOAL

Vuole sfruttare al massimo le sue capacità, e non ammette nessun ostacolo, pretende che gli strumenti a sua disposizione siano sempre efficienti e mai di intralcio.

INTERESSI

Adora il suo lavoro e ama la musica. Negli orari in cui non lavora, mentre ascolta musica classica, le piace avere massimo controllo su quello che i suoi potenziali clienti stanno facendo: vuole poter controllare le offerte in massima comodità, sfruttando i tempi morti per raggiungere i risultati ottimali.

Silvana
Agente Immobiliare

Figura 1.3: Esempio di *Persona* rappresentativa di un'agente immobiliare

1.1.3 Amministratori

La categoria con più privilegi è costituita dai gestori delle agenzie immobiliari. Dotati di accesso a funzionalità avanzate, hanno la possibilità di abilitare altri gestori e agenti immobiliari all'utilizzo della piattaforma.



Figura 1.4: Esempio di *Persona* rappresentativa di un amministratore

1.2 Modellazione dei casi d'uso

Individuati gli utenti target, si descrivono e schematizzano di seguito le funzionalità di DietiEstates25 e gli attori che ne prendono parte.

1.2.1 Descrizione dei casi d'uso

Gli **utenti** sono i principali attori della piattaforma, se ancora sono **non registrati** hanno la possibilità di:

- **Registrarsi tramite email:** L'utente compila un form con le informazioni richieste per la creazione di un account e il sistema verifica la validità dei dati inseriti, creando il nuovo profilo.
- **Registrarsi tramite Google:** L'utente si registra utilizzando il proprio account Google. Viene reindirizzato alla pagina di autenticazione di Google, dove concede i permessi necessari.

Una volta **registrati**, possono:

- **Autenticarsi tramite email:** L'utente inserisce le proprie credenziali nel form di login. Il sistema verifica la correttezza delle informazioni e, in caso positivo, concede l'accesso alla sua area riservata.
- **Autenticarsi tramite Google:** L'utente accede utilizzando il proprio account Google. Viene reindirizzato alla pagina di autenticazione di Google, dove concede i permessi necessari.

- **Ricerca immobili:** L'utente utilizza i filtri disponibili per cercare immobili. Il sistema elabora la richiesta e presenta i risultati corrispondenti ai parametri specificati.
- **Fare un'offerta su un immobile:** L'utente, selezionato un immobile, fa un'offerta per acquistarlo. Il sistema registra l'offerta e notifica l'agente immobiliare che ha caricato l'immobile.
- **Visualizzare immobili ricercati:** L'utente accede ad una sezione contenente la cronologia degli immobili visualizzati di recente.
- **Accettare un'offerta:** L'utente visualizza la controproposta ricevuta riguardo un'offerta fatta in precedenza e la accetta. Il sistema aggiorna lo stato dell'offerta e invia una notifica e-mail all'agente immobiliare che ha inviato la controproposta.
- **Rifiutare un'offerta:** L'utente esamina la controproposta ricevuta riguardo un'offerta fatta in precedenza e la rifiuta. Il sistema aggiorna lo stato dell'offerta e notifica l'agente immobiliare che ha inviato la controproposta.
- **Fare una controproposta ad un'offerta:** L'utente seleziona un'offerta ricevuta dall'agente immobiliare e propone una controproposta. Il sistema registra la controproposta e invia una notifica all'agente immobiliare.
- **Visualizzare le offerte fatte:** L'utente accede a una sezione dove può consultare l'elenco cronologico di tutte le offerte effettuate o ricevute, con i relativi dettagli.
- **Prenotare una visita per un immobile:** L'utente seleziona un immobile e sceglie una data e un orario per la visita nelle successive due settimane, compatibilmente con le disponibilità dell'agente. Il sistema registra la prenotazione e invia una notifica all'agente immobiliare.
- **Visualizzare le visite prenotate:** L'utente accede a una sezione dove può vedere l'elenco delle visite prenotate, con i dettagli relativi a ciascuna di esse.

Gli **agenti immobiliari**, oltre alle funzionalità degli **utenti**, possono:

- **Caricare un annuncio:** L'agente immobiliare inserisce le immagini e le informazioni relative ad un immobile da vendere tramite un apposito form. Il sistema convalida le informazioni e pubblica l'annuncio sulla piattaforma.
- **Inserire un'offerta dall'esterno:** L'agente immobiliare registra manualmente un'offerta ricevuta all'esterno della piattaforma. Il sistema la registra nello storico.
- **Accettare una prenotazione:** L'agente immobiliare visualizza le proposte di prenotazione e accetta una di esse. Il sistema aggiorna lo stato della visita e invia una notifica all'utente che l'ha richiesta.
- **Rifiutare una prenotazione:** L'agente immobiliare esamina le proposte di prenotazione e rifiuta una di esse. Il sistema aggiorna lo stato della visita e notifica l'utente che l'ha richiesta.

Infine, i **gestori**, oltre alle funzionalità degli **agenti**, possono:

- **Modificare la password di amministrazione:** Il gestore accede alle impostazioni dell'account e modifica la propria password. Il sistema verifica e salva la nuova password.
- **Creare account di supporto amministrazione:** Il gestore crea un nuovo account per nuovi amministratori. Il sistema genera le credenziali e imposta i permessi relativi.
- **Creare account per agenti immobiliari:** Il gestore crea un profilo per un nuovo agente immobiliare. Il sistema genera le credenziali e imposta i permessi relativi.

1.2.2 Diagramma dei casi d'uso

Tutti i casi d'uso individuati possono essere riassunti in un *Use Case Diagram* UML.



1.3 Descrizione dei requisiti non funzionali e di dominio

Descritti i requisiti funzionali di DietiEstates25, è necessario specificare i requisiti non funzionali e i requisiti di dominio, altrettanto importanti.

1.3.1 Requisiti non funzionali

I requisiti non funzionali fanno riferimento a caratteristiche del sistema come qualità, prestazioni, affidabilità e sicurezza.

Il Committente richiede che l'applicazione sia performante ed affidabile, attraverso cui gli utenti possano fruire delle funzionalità in modo intuitivo, rapido e piacevole. Tali richieste possono essere suddivise in più sezioni e tradotte nel seguente modo.

Requisiti di performance

La performance riguarda la capacità del sistema di elaborare e restituire informazioni in modo rapido, efficiente e fluido, garantendo tempi di risposta minimi. Assumendo condizioni di connettività ad internet ottimali, si individuano e rispettano i seguenti requisiti:

- L'applicazione deve garantire un caricamento delle pagine rapido e immediato, non superiore a 2 secondi.
- Il sistema deve restituire risultati alla ricerca di immobili in modo istantaneo, con un tempo massimo di 5 secondi.
- Il caricamento degli immobili deve avvenire velocemente, entro 10 secondi dalla conferma.

Requisiti di affidabilità

L'affidabilità misura la capacità del sistema di funzionare correttamente e continuativamente, gestendo eventuali errori senza perdita di dati o interruzione del servizio. Si individuano e rispettano i seguenti requisiti:

- L'applicazione deve garantire una disponibilità del servizio pressoché totale, con un uptime non inferiore al 99.5%.
- In caso di malfunzionamento, il sistema deve ripristinarsi completamente in meno di 10 secondi.
- Il sistema deve gestire eventuali errori in modo trasparente, senza interrompere l'esperienza di navigazione dell'utente se non in casi di malfunzionamento critico.
- L'applicazione deve possedere meccanismi di ripristino che consentano il recupero completo dei dati in caso di guasto.

Requisiti di usabilità

L'usabilità valuta la facilità di utilizzo dell'applicazione, misurando quanto l'interfaccia sia intuitiva, comprensibile e piacevole per l'utente finale. Si individuano e rispettano i seguenti requisiti:

- L'interfaccia utente deve consentire di completare qualsiasi azione principale con non più di 4 click.
- L'applicazione deve adattarsi perfettamente a schermi di qualsiasi dimensione, garantendo una visualizzazione ottimale.
- L'applicazione deve rispettare le linee guida di accessibilità WCAG livello AA.
- Il design deve garantire chiarezza assoluta di bottoni e menu, rendendone immediata la comprensione.

Requisiti di sicurezza

La sicurezza riguarda la protezione del sistema e dei dati degli utenti da manomissioni e violazioni della privacy. Si individuano rispettano i seguenti requisiti:

- L'applicazione deve implementare una comunicazione sicura che protegga totalmente i dati personali degli utenti.
- Il sistema deve possedere protezioni avanzate contro ogni tipo di attacco informatico.
- L'applicazione deve conservare le password degli utenti in forma crittografata e sicura anche in caso di violazione dei dati.
- Il sistema deve essere totalmente conforme alle normative GDPR per la protezione dei dati personali.
- L'applicazione deve chiudere automaticamente le sessioni dopo 15 minuti di inattività per prevenire accessi non autorizzati.

1.3.2 Requisiti di dominio

I requisiti di dominio di DietiEstates25 sono specifiche derivanti dalla conoscenza del contesto operativo e professionale del settore immobiliare. Rappresentano l'insieme degli standard e delle normative tipiche del settore e vanno necessariamente incorporate nel sistema. Si individuano rispettano i seguenti requisiti:

- Il sistema deve raccogliere esclusivamente i dati personali strettamente necessari, in conformità con l'Art. 5 del GDPR che prevede la minimizzazione dei dati.
- L'applicazione deve limitare la raccolta di informazioni solo agli elementi essenziali per la gestione dell'annuncio immobiliare.
- Deve essere implementata un'informativa privacy chiara e comprensibile, nel rispetto dell'Art. 13 del GDPR.
- L'applicazione deve implementare controlli di base per verificare la completezza e la coerenza dei dati inseriti negli annunci immobiliari.
- Il sistema fornirà linee guida chiare per la compilazione degli annunci, richiamando i principi di trasparenza previsti dal Codice Civile in materia di compravendita.
- Le credenziali di accesso saranno protette mediante tecniche di hashing, in linea con le best practice di sicurezza informatica.
- L'applicazione limiterà l'accesso ai dati personali, garantendo la protezione delle informazioni sensibili.

1.4 Esempi di casi d'uso

Per illustrare meglio il funzionamento del sistema, si formalizzano di seguito alcuni esempi di casi d'uso, secondo una descrizione testuale strutturata, realizzata utilizzando i template di Cockburn, e secondo la prototipazione visuale via Mock-up delle relative interfacce utente, tramite lo strumento di rapid prototyping Figma.

1.4.1 Caso d'uso #1:**Template di Cockburn**

Use Case #N	Name		
Goal in Context	This is a very long line. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.		
Preconditions			
Success End Conditions			
Failed End Conditions			
Primary Actor			
Trigger			
Description	Step	User Action	System
Extensions	Step	User Action	System
Subvariations	Step	User Action	System
Notes			

Mock-up**1.4.2 Caso d'uso #2:****Template di Cockburn**

Use Case #N	Name		
Goal in Context	This is a very long line. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.		
Preconditions			
Success End Conditions			
Failed End Conditions			
Primary Actor			
Trigger			
Description	Step	User Action	System
Extensions	Step	User Action	System
Subvariations	Step	User Action	System
Notes			

Mock-up**1.4.3 Caso d'uso #3:****Template di Cockburn**

Use Case #N	Name		
Goal in Context	This is a very long line. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.		
Preconditions			
Success End Conditions			
Failed End Conditions			
Primary Actor			
Trigger			
Description	Step	User Action	System
Extensions	Step	User Action	System
Subvariations	Step	User Action	System
Notes			

Mock-up**1.4.4 Caso d'uso #4:****Template di Cockburn**

Use Case #N	Name		
Goal in Context	This is a very long line. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.		
Preconditions			
Success End Conditions			
Failed End Conditions			
Primary Actor			
Trigger			
Description	Step	User Action	System
Extensions	Step	User Action	System
Subvariations	Step	User Action	System
Notes			

Mock-up**1.5 Glossario**

Sono di seguito definiti alcuni termini che sono stati utilizzati nel corso del capitolo.

- **Figma:** Strumento di progettazione di interfacce utente e prototipazione basato sul web, utilizzato per creare i mock-up.
- **GDPR (General Data Protection Regulation):** Regolamento europeo che disciplina il trattamento dei dati personali e la protezione della privacy.
- **Hashing:** Processo che trasforma dati di lunghezza arbitraria in una stringa di lunghezza fissa; usato per memorizzare password in modo non reversibile.
- **Mock-up:** Rappresentazione grafica e non funzionante dell'interfaccia utente, usata per validare layout e flussi prima dell'implementazione.
- **Persona:** Profilo sintetico di un utente tipo (obiettivi, bisogni, contesto) usato per guidare scelte di design e priorità funzionali.
- **UML (Unified Modeling Language):** Linguaggio standard per la modellazione di sistemi software (diagrammi di casi d'uso, classi, sequenze, ecc.).

- **Use Case (Caso d'uso):** Descrizione di una sequenza di interazioni tra attori e sistema che raggiungono uno scopo utile; usato per definire requisiti funzionali.
- **Uptime:** Percentuale di tempo in cui il servizio è disponibile e funzionante.
- **WCAG AA:** Linee guida per l'accessibilità dei contenuti web; il livello AA indica requisiti di accessibilità intermedicamente stringenti.

Capitolo 2

Progettazione del Sistema

2.1 Obiettivi di Progettazione

TODO Tra gli attributi di qualità del software, il sistema è stato progettato per garantire la piena **funzionalità** e **usabilità** da parte degli utenti finali, senza mai ignorare la **manutenibilità** e la **scalabilità**. L'**affidabilità**, l'**efficienza**, la **sicurezza** e l'**accessibilità**, seppur importanti e non trascurate, non sono state considerate altrettanto cruciali nella fase di progettazione.

2.2 Architettura del Sistema

Il sistema è stato progettato seguendo un'architettura a tre livelli: livello di presentazione, livello logico e livello di dati. Ogni livello è stato progettato per essere indipendente dagli altri, consentendo una maggiore flessibilità e facilità di manutenzione.

Il livello di presentazione è responsabile dell'interfaccia utente e della gestione delle interazioni con l'utente. È stato progettato per essere intuitivo e facile da usare, con un design responsivo che si adatta a diversi dispositivi e che garantisce accessibilità.

Il livello logico gestisce la logica di business del sistema, elaborando le richieste degli utenti e interagendo con il livello di dati. È stato progettato per essere scalabile e modulare, consentendo l'aggiunta di nuove funzionalità senza influire sulle parti esistenti del sistema.

Il livello di dati è responsabile della gestione e dell'archiviazione delle informazioni. È stato progettato per garantire l'integrità e la sicurezza dei dati, con prestazioni ragionevoli e una struttura scalabile, in modo da supportare la crescita del sistema.

Inoltre, sono state adottate tecnologie moderne e best practice di sviluppo software per garantire che il sistema sia robusto, sicuro e performante. Tra queste, l'uso di framework professionali per lo sviluppo, la gestione del versionamento del codice e l'implementazione di test automatici per garantire la qualità del software.

Di seguito, vengono illustrate in dettaglio le scelte progettuali e tecnologiche adottate per ciascun livello del sistema.

2.2.1 Front-End

Il front-end del sistema è progettato come applicazione mobile multiplatforma, sviluppata in **React Native** con **TypeScript**. Questa scelta tecnologica è motivata dalla necessità di garantire:

- **portabilità**, permettendo l'esecuzione su dispositivi iOS e Android tramite un unico codebase;
- **elevata manutenibilità**, grazie al typing statico fornito da TypeScript;
- **buona efficienza** in termini di prestazioni e reattività dell'interfaccia;
- **rapidità di sviluppo**, resa possibile dall'ecosistema React e dalla disponibilità di componenti già ottimizzati per dispositivi mobili.

L'architettura adottata segue i principi di **separation of concerns**, **scalabilità** e **testabilità**, strutturando l'applicazione in strati logici chiaramente definiti. Tale scelta consente di minimizzare l'accoppiamento tra componenti, facilitare l'evoluzione del sistema e supportare la manutenzione a lungo termine.

Strato di Presentazione (Presentation Layer)

Questo livello contiene tutti i componenti relativi alla visualizzazione e all'interazione con l'utente. I principi di design adottati seguono linee guida di **usabilità**, **consistenza** e **responsività**.

- **Schermate principali**: rappresentano le viste dell'app, strutturate in modo da favorire la comprensione immediata delle funzionalità.
- **Componenti UI riutilizzabili**: progettati per garantire uniformità visiva e ridurre la ridondanza.
- **Stili e risorse grafiche globali**: definiscono il design system dell'applicazione.

Le scelte di design dell'interfaccia utente privilegiano semplicità, chiarezza e accessibilità, con l'obiettivo di ridurre la curva di apprendimento e migliorare l'esperienza dell'utente finale.

Strato Logico e di Gestione dello Stato (Logic and State Management Layer)

In questo livello risiede la logica applicativa non direttamente legata alla visualizzazione. Esso è progettato secondo criteri di **modularità** e **riuso**.

- **Moduli di logica riutilizzabili**: incapsulano comportamenti specifici del dominio.
- **Contesti globali di stato**: gestiscono informazioni condivise tra più schermate.
- **Validazione dei dati**: garantisce la correttezza degli input del sistema.

Strato dei Servizi (Service Layer)

Questo strato gestisce la comunicazione con il back-end attraverso API RESTful. La scelta di separare questo livello facilita i test, riduce l'accoppiamento e consente di isolare cambiamenti nelle API.

- **Client API**: implementano le chiamate al back-end.
- **Servizi di business**: orchestrano operazioni complesse lato client.
- **DTO**: definiscono i formati dei dati scambiati con il server.

Strato di Persistenza Locale (Persistence Layer)

Il front-end utilizza un meccanismo di persistenza locale per migliorare l'esperienza utente, ridurre la latenza e supportare funzionalità offline, tramite la libreria AsyncStorage.

- **Repository:** astrazioni per l'accesso a dati remoti o locali.
- **Gestione sicura dei token:** garantisce la protezione di credenziali sensibili, attraverso la libreria Expo SecureStore.

Strato di Navigazione (Navigation Layer)

La navigazione è organizzata secondo un modello gerarchico e dichiarativo, che permette di gestire flussi complessi e protezione delle rotte.

- **Router logico:** definisce le relazioni tra schermate.
- **Protezione delle rotte:** limita l'accesso alle sezioni riservate agli utenti autenticati.
- **Pattern di navigazione:** supporto a tab, stack o navigazione annidata.

L'architettura descritta consente una netta separazione delle responsabilità, supporta la crescita del progetto e assicura un'elevata qualità del codice.

2.2.2 Back-End

Il back-end del sistema è sviluppato in **Java** utilizzando il framework **Spring Boot**, scelto per la sua architettura solida, scalabile e modulare. Tale framework fornisce un insieme di funzionalità integrate, quali gestione delle dipendenze, configurazione automatica e meccanismi di sicurezza preconfigurati, che consentono di ridurre la complessità infrastrutturale e di concentrare l'attenzione sulla logica applicativa e sulle esigenze del dominio.

Spring Boot supporta nativamente la creazione di **API RESTful**, che costituiscono l'interfaccia di comunicazione tra il front-end, sviluppato in **React Native**, e il back-end. L'architettura adottata segue un approccio multilivello (Controller-Service-Repository), che garantisce una chiara separazione delle responsabilità, migliorando manutenibilità e testabilità.

Strato di Presentazione (Controller Layer)

Questo livello espone le funzionalità applicative tramite endpoint REST:

- **Controller REST:** gestiscono le richieste HTTP e restituiscono risposte strutturate.
- **DTO (Data Transfer Object):** assicurano lo scambio sicuro e controllato di dati con il front-end.
- **Gestione centralizzata degli errori:** uniforma il formato delle risposte in caso di eccezioni.

Strato di Logica di Business (Service Layer)

Comprende le regole applicative e il coordinamento tra presentazione e persistenza:

- **Servizi di business:** implementano le operazioni del dominio.
- **Validazione:** garantisce la correttezza e la coerenza dei dati in ingresso.
- **Mapper:** trasformano automaticamente DTO ed entità di dominio.

Strato di Persistenza (Repository Layer)

Per la gestione dei dati è impiegato un database relazionale **PostgreSQL**. La persistenza è implementata tramite **Hibernate** come framework ORM, integrato con la **Jakarta Persistence API (JPA)**:

- **Entità JPA**: modellano le tabelle del database.
- **Repository Spring Data JPA**: permettono l'accesso ai dati tramite interfacce e query astratte.
- **Hibernate ORM**: gestisce la conversione tra oggetti Java e rappresentazione relazionale, riducendo la complessità operativa.

Questo approccio offre un alto livello di astrazione, migliorando portabilità e coerenza tra livello applicativo e schema dati.

Strato di Sicurezza

Il sistema di sicurezza è implementato tramite **Spring Security**, che garantisce un modello robusto di autenticazione e autorizzazione:

- **Filtri di sicurezza**: applicano controlli sugli endpoint sensibili.
- **Autenticazione JWT**: consente un approccio stateless, scalabile e sicuro.
- **Gestione ruoli e permessi**: definisce i livelli di accesso alle varie funzionalità.

Gestione dei Contenuti Multimediali

Per la memorizzazione e gestione di contenuti multimediali (come immagini o file di documentazione) è utilizzato **Azure Blob Storage**, una soluzione di archiviazione a oggetti scalabile e sicura. Il back-end integra questo servizio tramite SDK ufficiali, consentendo:

- caricamento e download dei file;
- generazione di URL sicuri e temporanei;
- gestione efficiente di contenuti di grandi dimensioni.

Deployment e Infrastruttura

Il processo di deployment è basato su una pipeline containerizzata:

- **Docker**: consente la creazione di container isolati e riproducibili per il back-end.
- **Microsoft Azure**: piattaforma cloud utilizzata per la distribuzione, garantisce scalabilità, alta disponibilità e monitoraggio continuo.
- **Ambienti separati** (sviluppo, test, produzione): supportano una gestione strutturata del ciclo di vita dell'applicazione.

Testing

La qualità e la stabilità del sistema sono assicurate tramite test automatici eseguiti con **JUnit**, che permettono:

- verifica delle unità funzionali;
- individuazione rapida di regressioni;
- supporto all'integrazione continua.

L'architettura descritta garantisce robustezza, scalabilità e una chiara separazione delle responsabilità, assicurando al sistema un'elevata manutenibilità e predisposizione all'evoluzione futura.

2.2.3 Database

Il database è progettato per essere scalabile e performante, utilizzando PostgreSQL.

PostgreSQL è scelto per la sua robustezza, e l'ottimo supporto alla gestione dei dati geospaziali (indispensabili per alcune funzionalità dell'applicazione).

La struttura del database è progettata per ottimizzare le prestazioni delle query senza mai compromettere l'integrità dei dati, anche impiegando l'indicizzazione.

Inoltre, sono implementate misure di sicurezza per proteggere i dati sensibili, come l'hashing dinamico delle password (tramite **BCrypt**).

Capitolo 3

Testing e Valutazione dell'Usabilità

Per garantire la qualità del sistema DietiEstates25, sono stati implementati test automatici e condotte valutazioni di usabilità.

I test automatici sono stati realizzati utilizzando JUnit per il back-end e Jest e React Testing Library per il front-end, coprendo le funzionalità principali e verificando la correttezza del comportamento del sistema.

La valutazione dell'usabilità è stata condotta attraverso ispezioni sistematiche e test con utenti volontari interessati all'applicazione.

Inoltre sono stati impiegati SonarQube per l'analisi della qualità del codice, individuando potenziali problemi e aree di miglioramento.

I risultati dei test automatici e delle valutazioni di usabilità sono stati analizzati per identificare eventuali problemi e apportare le necessarie correzioni al sistema, al fine di garantire un'esperienza utente fluida, intuitiva, accessibile mantenendo un elevato livello di affidabilità.

Di seguito, vengono presentati i dettagli dei test automatici e della valutazione dell'usabilità condotta.

3.1 Test Automatici

I test automatici sono stati implementati per garantire la qualità del codice e il corretto funzionamento del sistema.

A seguire, sono analizzate le tecniche di progettazione e gli strumenti utilizzati per il testing del back-end.

Strumenti Utilizzati

Per la stesura dei test è stato utilizzato il framework per applicazioni Java **JUnit 5**. Esso fornisce annotazioni e asserzioni per scrivere ed eseguire test in modo strutturato.

Per poter testare classi in isolamento, è stato utilizzato il framework di mocking per Java **Mockito**. Il suo scopo è creare "oggetti finti" (mock) di dipendenze esterne di una classe. Ciò permette di testare classi con dipendenze complesse senza doverle effettivamente istanziare.

Esempi

Nelle sezioni seguenti, vengono analizzati quattro esempi di metodi testati non banali e con almeno due parametri, illustrando le strategie adottate per la progettazione, tra cui le classi di equivalenza individuate e coperte e i criteri di copertura strutturale per ogni singolo esempio.

3.1.1 Metodo 1: createPropertyWithImages

Contesto

- **Metodo testato:** `createPropertyWithImages(CreatePropertyRequest request, List<MultipartFile> images)`
- **Scopo:** Creare una nuova proprietà caricando le immagini fornite, validandole, caricandole su Azure Blob Storage e salvando i dati della proprietà nel database. In caso di errore nella persistenza, il metodo esegue una logica di compensazione eliminando le immagini già caricate.
- **Test case:** `createPropertyWithImages_dbSaveFails_compensatesAndThrowsRuntimeException`

Strategia

Il test verifica il comportamento del sistema in presenza di un errore nella fase di persistenza su database. L'obiettivo non è validare un output, ma assicurarsi che il flusso in condizioni di fallimento attivi correttamente la logica di compensazione e propaghi l'eccezione prevista.

- **Classi di equivalenza individuate:** Le classi di equivalenza sono definite sui parametri del metodo:
 - **request:**
 - * CE-R1 (valida): request è non null e contiene tutti i campi necessari (classe valida del DTO).
 - * CE-R2 (non valida): request è malformata → la validazione solleva un'eccezione.
 - * CE-R3 (null): request = null → validazione fallisce immediatamente.
 - **images:**
 - * CE-I1 (lista vuota): `images.size() == 0`.
 - * CE-I2 (lista con immagini valide): ogni elemento passa la `validateImage()`.
 - * CE-I3 (lista con almeno un'immagine non valida): la validazione solleva eccezione.
 - * CE-I4 (lista null): `images == null`.
- **Classi coperte dal test:**
 - CE-R1 e CE-I2: request valida e immagini valide.
- **Copertura strutturale:** Il test esercita il ramo di gestione dell'eccezione generata durante la fase di persistenza, verificando l'esecuzione del blocco `catch` che implementa la compensazione. Questo garantisce la copertura del ramo alternativo (**branch coverage**) associato al fallimento del salvataggio su database.
- **Verifica:** La correttezza del comportamento è accertata tramite:
 - `assertThrows()`, che conferma la propagazione della `PersistenceException` generata dal fallimento della persistenza;
 - `verify(uploadImages)`, che attesta che l'upload è stato effettivamente tentato prima dell'errore;
 - `verify(deleteImages)`, che verifica che la logica di compensazione sia stata eseguita a seguito dell'eccezione.

3.1.2 Metodo 2: `getAgentOffers`

Contesto

- **Metodo testato:** `getAgentOffers(Long agentId, Pageable pageable)`
- **Scopo:** Recuperare una pagina di offerte associate a un agente specifico, arricchendo i dati con informazioni sulla proprietà e sull'utente.
- **Test case:** `testGetAgentOffers_Success`

Strategia

Il test verifica lo stato restituito dal servizio, assicurandosi che i dati della pagina di offerte siano completi, corretti e correttamente mappati nei DTO di risposta, incluse le relazioni verso utente e proprietà.

- **Classi di equivalenza individuate:** Le classi di equivalenza sono definite sui parametri del metodo:
 - **agentId:**
 - * CE-A1: `agentId` valido e corrispondente a un agente presente nel sistema.
 - * CE-A2: `agentId` non valido o inesistente, cioè la query restituisce una pagina vuota.
 - * CE-A3: `agentId` nullo.
 - **pageable:**
 - * CE-P1: paginazione valida (dimensione pagina e indice validi).
 - * CE-P2: paginazione con dimensione zero o negativa.
- **Classi coperte dal test:**
 - CE-A1 – `agentId` valido
 - CE-P1 – paginazione valida
- **Verifica:** La correttezza del metodo è accertata tramite:
 - `assertNotNull(result)` e `assertEquals(size, ...)` per validare la pagina e il numero di offerte.
 - Asserzioni sui singoli elementi della pagina per verificare:
 - * correttezza degli ID delle offerte,
 - * presenza e correttezza delle relazioni verso utente (`User`) e proprietà (`Property`),
 - * coerenza dei dati rispetto al mock configurato.
 - `verify(offerRepository, times(1))` per garantire che la query verso il repository sia stata effettivamente invocata.

3.1.3 Metodo 3: calculateBoundingBox

Contesto

- **Metodo testato:** `calculateBoundingBox(BigDecimal centerLatitude, BigDecimal centerLongitude, double radiusMeters)`
- **Scopo:** Calcolare il bounding box di un cerchio definito da una latitudine, longitudine e raggio in metri, considerando il caso particolare del passaggio dell'antimeridiano.
- **Test case:** `testAntimeridianCrossingAndContainment`

Strategia

Il test è un test di unità puro, focalizzato sui valori limite e sui casi particolari geografici. L'obiettivo è verificare sia la correttezza numerica del bounding box sia la gestione dei punti rispetto al passaggio dell'antimeridiano.

- **Classi di equivalenza:** Le classi di equivalenza sono definite sui parametri del metodo:
 - **centerLatitude:**
 - * CE-L1: latitudine valida compresa tra -90° e 90° .
 - * CE-L2: latitudine ai limiti $\pm 90^\circ$.
 - * CE-L3: latitudine nulla o fuori range.
 - **centerLongitude:**
 - * CE-Lo1: longitudine valida compresa tra -180° e 180° .
 - * CE-Lo2: longitudine vicino ai limiti $\pm 180^\circ$ (test passaggio antimeridiano).
 - * CE-Lo3: longitudine nulla o fuori range.
 - **radiusMeters:**
 - * CE-R1: raggio positivo e non nullo.
 - * CE-R2: raggio nullo o negativo.
- **Classi coperte dal test:**
 - CE-L1 – latitudine valida all'equatore (0°)
 - CE-Lo2 – longitudine vicina a 180° per verificare il passaggio dell'antimeridiano
 - CE-R1 – raggio positivo sufficiente a coprire il passaggio dell'antimeridiano
- **Strategia di verifica:** La correttezza è verificata attraverso:
 - Controllo del bounding box generato, in particolare `minLon > maxLon` per rilevare il passaggio dell'antimeridiano.
 - Verifica che punti all'interno del raggio (vicino a $\pm 179.95^\circ$) siano inclusi nel bounding box (`isPointInBoundingBox()`).

3.1.4 Metodo 4: `getAgentVisits`

Contesto

- **Metodo testato:** `getAgentVisits(Long agentId, Pageable pageable)`
- **Scopo:** Restituire una pagina di visite associate a un agente specifico, arricchendo i dati con informazioni sulla proprietà, indirizzo e utente, gestendo la paginazione.
- **Test case:** `testGetAgentVisits_ReturnsPage`

Strategia

Il test verifica l'interazione tra il servizio e il repository, assicurandosi che la pagina restituita contenga le informazioni corrette e che il repository venga interrogato con i parametri appropriati.

- **Classi di equivalenza:** Le classi di equivalenza sono definite sui parametri del metodo:
 - **agentId:**
 - * CE-A1: **agentId** valido corrispondente a un agente esistente.
 - * CE-A2: **agentId** non valido o inesistente → restituzione di pagina vuota.
 - * CE-A3: **agentId** nullo → comportamento dipendente dal servizio/repository (potenziale eccezione).
 - **pageable:**
 - * CE-P1: paginazione valida (dimensione pagina e indice validi).
 - * CE-P2: paginazione con dimensione zero o negativa → gestione prevista dal framework.
- **Classi coperte dal test:**
 - CE-A1 – **agentId** valido
 - CE-P1 – paginazione valida
- **Copertura strutturale:** Il test copre l'unico percorso logico del metodo, garantendo la **statement coverage** dell'interazione tra servizio e repository.
- **Verifica:** La correttezza è accertata tramite:
 - `assertEquals(result, mockPage)` per confermare che il servizio restituisca la pagina attesa;
 - `verify(visitRepository).getAgentVisits(agentId, pageable)` per garantire che il repository sia stato invocato con i parametri corretti.

3.2 Valutazione dell'Usabilità

La valutazione dell'usabilità è un processo fondamentale per garantire che un sistema software non solo sia funzionale, ma anche efficace, efficiente e soddisfacente per l'utente finale. Per DietiEstates25, è stato adottato un approccio multi-metodo che combina ispezioni euristiche condotte da esperti (noi) con test di usabilità che coinvolgono utenti reali. Questo approccio ha permesso di raccogliere dati sia qualitativi che quantitativi, offrendo una visione completa dei punti di forza dell'applicazione.

3.2.1 Expert Review (Ispezione Euristica)

L'ispezione euristica è una tecnica di valutazione in cui uno o più esperti esaminano l'interfaccia utente e la confrontano con un insieme di principi di usabilità riconosciuti. Per questa valutazione, sono state utilizzate le 10 euristiche di usabilità di Nielsen-Molich, adattate al contesto di DietiEstates25.

Checklist Euristica e Risultati

La checklist è stata applicata sistematicamente a tutte le principali funzionalità dell'applicazione, dalla registrazione alla gestione delle offerte. I risultati sono riassunti nella tabella seguente.

Euristica	Risultati e Osservazioni per DietiEstates25
Dialoghi semplici e naturali	<i>Conforme.</i> L'interfaccia è pulita e priva di elementi superflui. Le informazioni sono presentate in modo chiaro, focalizzando l'attenzione sui contenuti rilevanti (es. dettagli dell'immobile, offerte).
Parlare la lingua dell'utente	<i>Conforme.</i> Il linguaggio utilizzato (es. "offerta", "visita", "annuncio") è familiare per il dominio immobiliare. Le icone sono intuitive e universalmente riconosciute dagli utenti.
Minimizzare il carico di memoria dell'utente	<i>Conforme.</i> Le informazioni importanti sono sempre accessibili, ad esempio nello storico, riducendo il carico cognitivo dell'utente.
Consistenza	<i>Conforme.</i> L'applicazione mantiene una forte consistenza visiva e comportamentale. I pulsanti di azione primaria sono sempre posizionati in modo simile e il design system è applicato uniformemente in tutte le schermate.
Feedback	<i>Conforme.</i> L'applicazione fornisce feedback costanti ma non invadenti sulle operazioni. Ad esempio, un indicatore di caricamento è sempre visibile durante le chiamate di rete e messaggi di conferma appaiono dopo azioni importanti come l'invio di un'offerta.
Uscite chiaramente definite	<i>Conforme.</i> Gli utenti possono sempre facilmente annullare operazioni come la compilazione di un form.
Shortcuts	<i>Non applicabile.</i> L'interfaccia è sufficientemente semplice per i neofiti. Non sono presenti scorciatoie per utenti esperti, ma data la natura dell'app non sono considerate essenziali.
Buoni messaggi di errore	<i>Conforme.</i> I messaggi di errore sono chiari e costruttivi. Ad esempio, se il login fallisce, il sistema specifica se il problema è la password o l'username, senza essere vago.
Prevenzione degli errori	<i>Conforme.</i> La validazione dei form previene molti errori comuni (es. formato email non valido). Ci sono inoltre controlli di conferma prima di azioni importanti.
Aiuto e documentazione	<i>Non applicabile.</i> Non è presente una sezione di aiuto dedicata. Tuttavia, la semplicità dell'applicazione e le etichette chiare rendono il suo utilizzo autoesplicativo, minimizzando la necessità di una documentazione formale per l'utente finale.

Tabella 3.1: Risultati dell'ispezione euristica.

3.2.2 Test con Utenti

Per integrare i risultati dell'analisi euristica, è stato condotto un esperimento con un piccolo gruppo di utenti potenziali rappresentativi delle principali fasce di età.

Soggetti Reclutati

Nel gruppo di test sono stati coinvolti 5 utenti, di età compresa tra i 20 e i 60 anni, con familiarità nell'uso di applicazioni mobile ma senza esperienza specifica nel settore immobiliare. Essi rappresentano un campione diversificato di utenti potenziali.

Procedura Sperimentale

A ciascun utente è stato chiesto di eseguire una serie di compiti sull'applicazione, utilizzando il protocollo *think-aloud* (pensare a voce alta) per verbalizzare i propri pensieri e le proprie difficoltà. Le sessioni, della durata media di 15 minuti, sono state osservate da un membro del team. I compiti assegnati (con rispettivi tempi di completamento) erano:

1. Effettuare la registrazione di un nuovo account come cliente.
2. Eseguire una ricerca per un immobile di tipo "Appartamento" con svariati filtri.
3. Selezionare un immobile dai risultati e visualizzarne i dettagli.
4. Prenotare una visita per l'immobile scelto.
5. Inviare un'offerta di acquisto per lo stesso immobile.

Metriche Considerate

Sono state raccolte le seguenti metriche:

- **Tasso di successo dei compiti:** Percentuale di utenti che ha completato con successo ciascun compito.
- **Tempo di completamento:** Tempo impiegato per portare a termine ogni compito.
- **Numero di errori:** Numero di azioni errate o vicoli ciechi incontrati.
- **Feedback soggettivo:** Commenti, critiche e suggerimenti raccolti durante il protocollo *think-aloud*.

Survey

Al termine della sessione, a ogni partecipante è stato chiesto di compilare un breve questionario di valutazione basato su una scala Likert (1 = Per niente d'accordo, 5 = Pienamente d'accordo). Di seguito sono riportate le affermazioni e i relativi punteggi medi raccolti:

- **Q1 (Semplicità d'uso):** Ho trovato l'applicazione semplice da usare. 4.6 / 5
- **Q2 (Necessità di supporto):** Penso che avrei bisogno di supporto tecnico per essere in grado di usare questa applicazione. 1.2 / 5
- **Q3 (Integrazione delle funzioni):** Ho trovato le varie funzioni in questa applicazione ben integrate tra loro. 4.8 / 5
- **Q4 (Piacevolezza grafica):** Ho trovato l'interfaccia graficamente piacevole. 4.8 / 5
- **Q5 (Comfort d'uso):** Mi sono sentito a mio agio nell'utilizzare l'applicazione. 5 / 5

Risultati finali

I risultati del test sono positivi. Tutti e 5 gli utenti (100%) sono riusciti a completare con successo tutti i compiti. Un solo utente meno pratico ha richiesto assistenza per i compiti 1 e 4.

I tempi di completamento medi brevi sono indice di un'interfaccia intuitiva e ben progettata.

I commenti qualitativi hanno evidenziato un apprezzamento generale per la pulizia dell'interfaccia e la fluidità della navigazione.

I risultati del survey, infine, confermano l'elevata usabilità percepita dagli utenti.

3.3 SonarQube

Lo strumento di analisi della qualità del codice SonarQube è stato utilizzato per il lato back-end di DietiEstates, individuando potenziali problemi e aree di miglioramento.

L'analisi ha permesso di identificare vulnerabilità, code smells e duplicazioni, contribuendo a mantenere un codice pulito, facilmente manutenibile e scalabile.

I risultati dell'analisi sono stati integrati nel processo di sviluppo, consentendo di apportare le necessarie correzioni e miglioramenti al codice.

Il report corrente suggerisce che non sono presenti issue aperte e che il Quality Gate è stato superato, come visibile attraverso [SonarCloud](#).

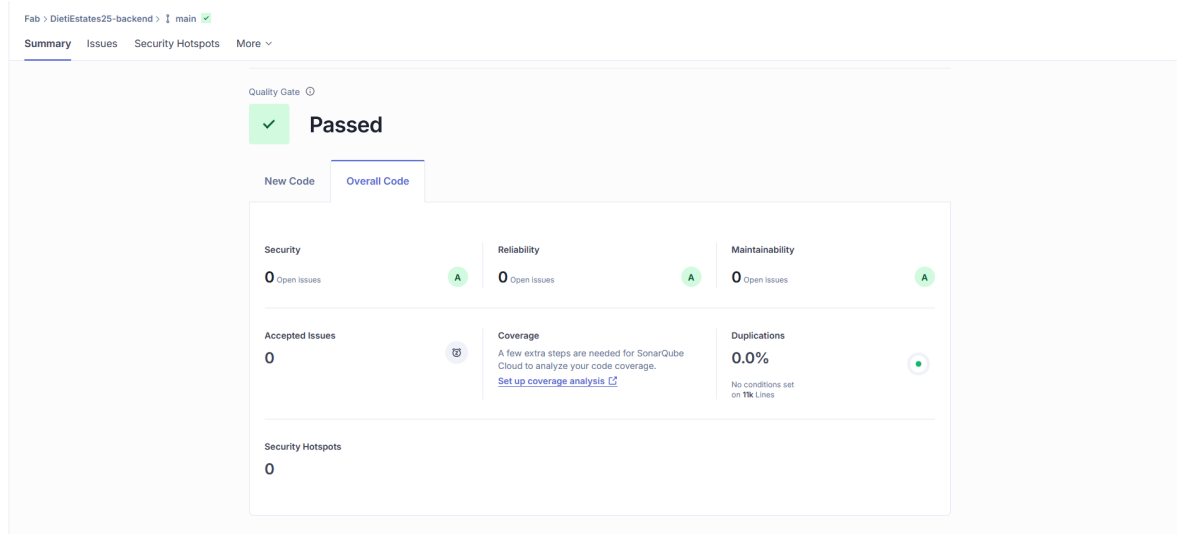


Figura 3.1: *Report* di SonarCloud per il back-end

3.4 Accessibilità

TODO Per garantire l'accessibilità dell'applicazione, sono state adottate le linee guida WCAG livello AA.

Sono stati usati strumenti automatici per verificare la conformità alle linee guida, ad esempio nella scelta dei colori, identificando e correggendo eventuali problemi di accessibilità.

TODO aggiungere palette TODO traduzioni

Appendice A

Evidenza di versioning

Il versioning del codice sorgente è stato gestito tramite **Git** con hosting del repository su **GitHub**. Git è un sistema di versionamento distribuito che registra in modo incrementale lo stato dei file tramite commit, mantenendo uno storico completo e permettendo branching e merging. Ogni copia locale è un repository completo. GitHub è invece una piattaforma remota che ospita repository Git, aggiungendo funzionalità di collaborazione come pull request e issue tracking. Il suo utilizzo ha permesso di tracciare la storia delle modifiche, facilitare la collaborazione tra i membri del team e integrare il controllo qualità con SonarQube.

E' possibile visualizzare i repository del [back-end](#) e del [front-end](#) pubblicamente.

Dati riepilogativi

TODO

- Numero di contributors:
- Numero di commit:
- Frequenza dei commit:

TODO sistemare impaginazione